NYATIFY · CHANGE SUMMARY

What changed from the original program to now

This note explains the work so far in very simple words. It starts from the first saved copy of Nyatify and ends at the current code, including voice-notify work that is not committed yet.

Project	Nyatify (Discord bot)
Date	26 August 2026
Original snapshot	First git commit (one small Python file)
Current stage	Main branch + extra voice-notify edits
Language	Plain English. Short sentences.

1. The short version

The original bot was one short script. It watched voice channels. When the first person joined an empty voice room, it pinged everyone in one hard-coded text channel.

The current bot is a small product. It still does that same main job. Around that job we added: separate features, per-server settings, admin commands, slash commands, safer config, tests, and smarter voice notices.

In one line: we kept the cute “someone started voice” message, and we rebuilt everything around it so it can grow, be tested, and be set up by each server.

2. What the original program was

Files

Only a handful of files:

- nyatify.py — all the bot logic in one place
- config.example.txt — only the Discord token
- requirements.txt — discord.py
- Dockerfile — run the bot in a container
- .gitignore / .dockerignore — keep the token off git

What it did

- Read DISCORD_TOKEN from the environment or config.txt.
- Turn on voice and member intents.
- Keep a set of “active” voice channel IDs.
- When someone joined and the room had 1 member, send @everyone plus a pink embed.
- When the last person left, forget that room so the next first join could ping again.

Limits (why we changed it)

- The text channel ID was written in the code. One channel for every server.
- Bots counted as people. A music bot joining could trigger a ping.
- No admin commands. No way to turn a feature on or off without editing code.
- A quick leave and rejoin could spam pings.
- AFK / timeout rooms were treated like normal voice rooms.
- No tests. Hard to change without breaking it.
- Restarting the bot forgot which rooms were already occupied.

3. How the project is shaped now

The bot is no longer one file. Each job lives in its own place. The main file only starts the bot and loads features.

Main files (what each one is for)

nyatify.py	Start the bot. Load features. Sync slash commands.
config.py	Read settings from env or config.txt.
loader.py	Find which feature files to load.
guild_config.py	Save each server's announce channels on disk.
announce.py	One shared way to post a notice in a server.
cogs/voice_notify.py	The voice start feature.
cogs/admin.py	Owner and admin commands.
tests/	Automated checks. No Discord login needed.

Why this shape

- A new feature can be a new file. We do not pile more into nyatify.py.
- We can load only one feature (ENABLED_COGS) when debugging.
- Voice notify posts through announce.py. It does not pick a channel by itself.
- Tests can fake Discord objects and check the rules.

4. Settings (config)

Original config had only the token. Current config can also set:

- COMMAND_PREFIX — default is still !
- LOG_LEVEL — how chatty the logs are
- ENABLED_COGS — empty means all features; a name means only that one
- SYNC_GUILD_ID — a server ID so /commands show up right away while testing

Environment variables still win over config.txt. If the token is missing, the bot stops with a clear error instead of crashing later.

5. Voice notify — behavior changes

This is still the main user-facing feature. The look of the message is almost the same: pink embed, "Hello Every Nya", @everyone. The rules behind it are much stricter.

When it announces

- A real person (not a bot) joins a voice or stage channel.
- The room had no other humans.
- The room is not the server AFK channel.
- We are not still inside the "please wait" window after everyone left.
- That person does not already have a live start-message we can update instead.

When it stays quiet

- Someone joins a room that already has humans.
- A bot joins.

- Someone only mutes, deafens, or streams — they did not change rooms.
- Someone hops rooms during the wait window after the last human left.
- No announce channel is set for that server (it may send a one-time setup hint instead).

Reconnect wait (45 seconds)

If the last human leaves, we do not treat the room as “empty for a new ping” right away. We wait 45 seconds. If someone comes back in that time, we do not ping again. This cuts spam from disconnects and accidental leaves.

The notice lives 15 minutes

Each start-message is remembered. After 15 minutes the bot tries to delete it from Discord. A background loop checks about every 30 seconds. IDs are also saved to a JSON file, so a restart can still delete old messages.

If the same person moves rooms

If that person already has a live start-message, we edit it to the new room instead of posting a second ping. The Channel field in the embed updates.

The embed itself

Original fields: User, Channel Occupancy (how many people).

Current fields: User, Channel (a clickable mention of the voice room).

We dropped occupancy because “first person in an empty room” is always one human.

Memory after restart

On ready, the bot looks at who is already in voice. Rooms that already have humans are marked active, so we do not ping again for rooms that were already going. If the bot leaves a server, or a voice room is deleted, we drop that memory.

Safety in the code

- A lock per voice room, so two joins at the same time do not double-ping.
- If sending the message fails, we do not keep the room marked as announced.
- Only humans are counted, never bots.

6. Each server picks its own announce channel

This is a big change. The original bot always posted to one channel ID in the source code. Now each Discord server stores its own settings in data/guild_config.json.

Two layers

- Default channel — every announce feature uses this.
- Per-function channel — for example voice can use a different channel.

If the voice-only channel was deleted, the bot falls back to the default channel and can send one short

hint so an admin can fix it. If nothing is set, it can send one setup hint in the system channel (or the first text channel it can use).

Who can change it

People with Manage Server. Prefix commands and slash commands both work.

Prefix examples

- !announce — show what is set
- !announce #channel — all functions use this channel
- !announce voice #channel — only voice uses this channel
- !announce clear — wipe all announce channels
- !announce clear voice — voice goes back to the default

Slash examples

- /announce show
- /announce set
- /announce feature
- /announce clear

7. Admin and slash commands

The original bot had no commands. Admins could not change anything from Discord. Now there is an admin feature with two kinds of commands.

For the bot owner

- /ping or !ping — check the bot is alive
- /cogs or !cogs — list loaded and available features
- /load, /unload, /reload — turn a feature on or off without restarting

The admin feature cannot be unloaded. That would lock you out of those commands.

Slash sync

On start, the bot copies commands to Discord. If SYNC_GUILD_ID is set, they appear in that test server immediately. If it is empty, Discord global sync can take a long time (up to about an hour).

Announce slash commands are guild-only. Discord also hides them from people who cannot Manage Server. Errors try to reply in simple words instead of a crash.

8. Tests

The original program had zero tests. Now there is a pytest suite. Tests fake Discord objects. They do not log into Discord.

- tests/cogs/test_voice_notify.py — many cases for joins, leaves, grace, TTL, edits, AFK, bots
- tests/cogs/test_admin.py — announce and owner commands
- tests/test_announce.py — picking a channel and sending hints

- tests/test_guild_config.py — saving and clearing server settings
- tests/test_config.py and test_loader.py — settings and which features load

There are also requirements-dev.txt and pytest.ini. This lets us change voice rules with less fear of silent breakage.

9. What we did not throw away

- It is still a discord.py bot.
- It still uses prefix “!” by default.
- It still needs voice_states and members intents.
- The ping still uses @everyone and the same pink color and title.
- Docker is still there to run the bot.
- The real token is still not supposed to be in git (config.txt is ignored).

10. Work in order (git timeline)

These are the saved steps on main, plus the extra files you have open now.

Snapshot 0 — original

One script. Hard-coded channel. Join empty room → ping. Leave empty → forget.

Commit 1 — split into features

Moved logic into cogs. Added config.py. Admin stub and a first voice_notify cog. nyatify.py became a loader, not the whole program.

Commit 2 — debug and tests

Added loader.py so we can run one feature alone. Added pytest, helpers, and first tests. Better logging and config keys.

Commit 3 — Voice_Notify

Real per-server announce channels. guild_config.py and announce.py. Admin !announce. Much richer voice rules and a large test file.

Commit 4 — Voice_Notify +

Cleanup of the same design. Clearer announce path. Less duplicated lookup code.

Commit 5 — Voice_Notify Slash

Slash /announce plus hybrid owner commands. SYNC_GUILD_ID. Prefix and slash share the same announce logic.

Not committed yet (current editor state)

Voice notify grew again: 45-second reconnect wait, 15-minute notice lifetime, save notices to JSON, delete expired messages, edit the live notice when the same person moves, and more tests for those

rules.

If you only look at the last git commit, you will miss this last voice-notify layer. This PDF treats that working copy as part of “current state.”

11. Side-by-side

Original

- One file, one channel ID, one feature.
- Bots can trigger a ping.
- No wait after empty. Easy to spam.
- Notices stay forever.
- No commands. No tests. No per-server setup.

Current

- Many files. Features load as plugins.
- Each server sets channels with !announce or /announce.
- Humans only. AFK ignored. 45s wait. Locks against double posts.
- Notices expire after 15 minutes and can be edited if the starter moves.
- Owner can load/unload features live. Hundreds of automated tests.

12. What this stage is for

The current stage is a solid first product: one real feature (voice start), a shared announce system ready for more features, and admin tools to run it in real servers.

A later stage can add more notify functions. They should post through announce.py, not copy-paste channel lookup.

Still true: the bot must have permission to mention @everyone and to talk in the announce channel. Someone with Manage Server still needs to run /announce set (or !announce) once per server.